

React Interview — FULL GUIDE

Installation → JSX → Components → Props/State → Hooks → Render/Re-render → Reconciliation → Routing → API → Forms → Zustand → Tricky savollar

Maqsad: javoblarni yodlash emas, ma’nosini tushunib og’zaki aytish. Har bir savolga “nima?”, “nima uchun?”, “farqi nima?” va “misol” follow-up savollariga tayyorlan.

0. Installation va Project Setup

□ React projectni qanday yaratamiz?

Javob: Vite bilan: `npm create vite@latest my-app`. Keyin `cd my-app`, `npm install` va `npm run dev`.

□ Vite nima?

Javob: Frontend projectni tez yaratish va development server/build jarayonini boshqarish uchun zamonaviy build tool.

□ npm install nima qiladi?

Javob: `package.json` dagi dependencylarni o’rnatadi va `node_modules` yaratadi.

□ npm run dev nima?

Javob: Development serverni ishga tushiradi.

□ react va react-dom farqi?

Javob: `react` asosiy React APIlarini beradi; `react-dom` React UI’ni browser DOMiga ulash uchun ishlatiladi.

□ package.json nima?

Javob: Project metadata, scripts va dependencylar ro’yxatini saqlaydigan fayl.

□ node_modules nima?

Javob: O’rnatilgan npm paketlari joylashadigan papka; odatda Gitga yuborilmaydi.

1. React va asosiy tushunchalar

□ React nima?

Javob: React — JavaScript asosida qurilgan UI library. Componentlar yordamida interaktiv user interface yaratish uchun ishlatiladi.

□ React nima uchun ishlatiladi?

Javob: Qayta ishlatiladigan componentlar, state va interaktiv UI orqali zamonaviy frontend applicationlar qurish uchun.

□ Library va framework farqi?

Javob: Libraryda dasturchi flow ustidan ko’proq nazorat qiladi; framework esa application structure va flowni ko’proq belgilaydi.

□ SPA nima?

Javob: Single Page Application — browser sahifani to’liq reload qilmasdan UI va route kontentini yangilaydigan application.

□ Component nima?

Javob: UI ning qayta ishlatiladigan kichik qismi.

□ Functional component nima?

Javob: JSX qaytaradigan JavaScript function.

□ Nega component nomi katta harf?

Javob: React lowercase nomlarni HTML tag, uppercase nomlarni custom component sifatida ajratadi.

2. JSX

□ JSX nima?

Javob: JavaScript XML. JavaScript ichida HTMLga o'xshash syntax bilan UI yozishga imkon beradi.

□ **JSX HTMLmi?**

Javob: Yo'q. JSX HTMLga o'xshash syntax; build jarayonida JavaScriptga transform qilinadi.

□ **JSXda class nega className?**

Javob: JSX React atributlarida className ishlatiladi.

□ **Expressionlar qanday yoziladi?**

Javob: JavaScript expressionlar {} ichida yoziladi: {name}.

3. Props

□ **Props nima?**

Javob: Parent componentdan child componentga data yoki qiymat uzatish uchun ishlatiladigan read-only obyekt.

□ **Propsga nimalar yuborish mumkin?**

Javob: String, number, boolean, array, object, function, JSX va boshqa qiymatlar.

□ **Child propsni o'zgartira oladimi?**

Javob: Propsni bevosita mutate qilmasligi kerak. O'zgartirish kerak bo'lsa parent callback yoki setter yuboradi.

□ **children nima?**

Javob: Component taglari orasidagi JSXni qabul qiladigan maxsus prop.

4. State va useState

□ **State nima?**

Javob: Componentning o'zgaruvchan holati. State yangilanganda React componentni qayta render qiladi.

□ **useState nima?**

Javob: State yaratish va yangilash uchun React Hook.

□ **const [count,setCount]=useState(0)?**

Javob: count — current state, setCount — setter, 0 — initial value.

□ **Previous state qachon?**

Javob: Yangi qiymat oldingi qiymatga bog'liq bo'lsa: setCount(prev => prev + 1).

□ **Props va State farqi?**

Javob: Props parentdan keladi va read-only; state component tomonidan boshqariladi va setter bilan yangilanadi.

5. Rendering, Re-render va Reconciliation

□ **Render nima?**

Javob: Component functionining bajarilib, Reactga kerakli UI natijasini hisoblatish jarayoni.

□ **Re-render nima?**

Javob: State, props yoki boshqa React triggerlari natijasida componentning yana render qilinishi.

□ **State o'zgarsa nima bo'ladi?**

Javob: React update qiladi, component qayta render bo'ladi va kerakli DOM o'zgarishlari commit qilinadi.

□ **Re-render real DOMni to'liq qayta chizadimi?**

Javob: Yo'q. React yangi va oldingi natijalarni taqqoslab, kerakli DOM o'zgarishlarini qo'llaydi.

□ **Reconciliation nima?**

Javob: Reactning oldingi va yangi render natijalarini solishtirib, Ulga kerakli o'zgarishlarni aniqlash jarayoni.

□ **Mount nima?**

Javob: Componentning Ulga birinchi marta qo'shilishi.

□ **Update nima?**

Javob: Componentning state/props/context kabi sabablar bilan yangilanishi.

□ **Unmount nima?**

Javob: Componentning UI daraxtidan olib tashlanishi.

□ **Virtual DOM nima?**

Javob: React UI modelining JavaScriptdagi representationi. React undan reconciliationda foydalanadi.

□ **Commit phase nima?**

Javob: Hisoblangan o'zgarishlarni DOMga qo'llash bosqichi.

6. useEffect

□ **useEffect nima?**

Javob: Side effectlar uchun Hook: API request, timer, event listener, document title va boshqalar.

□ **Dependency array nima?**

Javob: Effect qachon qayta ishlashini belgilaydi.

□ **[] nima?**

Javob: Effect odatda mountdan keyin bir marta ishlaydi; Strict Mode developmentda qo'shimcha tekshirishi mumkin.

□ **[count] nima?**

Javob: count o'zgargan render commitidan keyin effect qayta ishlaydi.

□ **Dependency array bo'lmasa?**

Javob: Effect har commitdan keyin ishlaydi.

□ **Cleanup nima?**

Javob: Timer, listener yoki subscription kabi resurslarni tozalash functioni.

7. useRef

□ **useRef nima?**

Javob: Renderlar orasida saqlanadigan mutable ref object yaratadi. current o'zgarsa re-render bo'lmaydi.

□ **Qayerda ishlatiladi?**

Javob: DOMga focus/scroll kabi murojaat va renderga ta'sir qilmaydigan qiymatlarni saqlashda.

□ **useState va useRef farqi?**

Javob: State o'zgarsa re-render; ref.current o'zgarsa re-render bo'lmaydi.

8. Events va Forms

□ **onClick?**

Javob: Click eventda function chaqirish.

□ **onChange?**

Javob: Input qiymati o'zgarganda handlarni chaqirish.

□ **onSubmit?**

Javob: Form yuborilganda handlarni chaqirish.

□ **preventDefault?**

Javob: Browser default submit/reload xatti-harakatini to'xtatadi.

□ **Controlled input?**

Javob: Input qiymati React state bilan boshqariladi.

□ **Uncontrolled input?**

Javob: Input qiymatini DOM boshqaradi; ref orqali olinishi mumkin.

9. Lists va key

□ map nima uchun?

Javob: Arraydan React elementlar ro'yxatini yaratish.

□ key nima?

Javob: List elementlariga stable identity berib, reconciliationda to'g'ri moslashtirish uchun.

□ Indexni key qilish mumkinmi?

Javob: Mumkin, lekin list tartibi o'zgarsa muammolar bo'lishi mumkin. Stable unique ID afzal.

□ Random key nega yomon?

Javob: Har renderda identity o'zgarib, component remount bo'lishi va local state reset bo'lishi mumkin.

10. Componentlar orasidagi aloqa

□ Parent → Child?

Javob: Props orqali.

□ Child → Parent?

Javob: Parent callback functionni prop qilib beradi, child uni chaqiradi.

□ Lifting state up?

Javob: Siblingsga kerak bo'lgan state'ni umumiy parentga ko'tarish.

□ Prop drilling?

Javob: Data kerak bo'lgan childgacha ko'p oraliq componentlar orqali props uzatish.

11. Context API

□ Context nima?

Javob: Component tree bo'ylab qiymatni ko'p qatlam props uzatmasdan yetkazish mexanizmi.

□ Provider nima?

Javob: Context value'ni descendant componentlarga beradi.

□ useContext nima?

Javob: Component ichida context value'ni olish Hooki.

12. useMemo, useCallback, memo

□ useMemo?

Javob: Hisoblangan qiymatni dependencylar o'zgarmaguncha memoize qiladi.

□ useCallback?

Javob: Function reference'ni dependencylar o'zgarmaguncha saqlaydi.

□ React.memo?

Javob: Propslar o'zgarmaganda ayrim keraksiz component re-renderlarini kamaytirishga yordam beradi.

□ useMemo vs useCallback?

Javob: useMemo — qiymat; useCallback — function reference.

□ Har doim kerakmi?

Javob: Yo'q. Real performance ehtiyoji bo'lsa ishlatiladi.

13. Routing

□ React Router?

Javob: React applicationda URL va UI navigation/routingni boshqarish vositasi.

□ Link?

Javob: Full page reloadsiz route o'zgartirish.

□ **useNavigate?**

Javob: Programmatic navigation.

□ **useParams?**

Javob: Dynamic URL parametrlarini olish.

□ **Nested route?**

Javob: Parent route ichidagi child route.

□ **Outlet?**

Javob: Nested child route render bo'ladigan joy.

14. API va Async

□ **API?**

Javob: Frontend va backend o'rtasida data almashish interfeisi.

□ **GET/POST/PUT/PATCH/DELETE?**

Javob: GET olish; POST yaratish; PUT/PATCH yangilash; DELETE o'chirish.

□ **fetch?**

Javob: Browsering HTTP request API'si.

□ **async/await?**

Javob: Promise asosidagi asynchronous kodni o'qilishi oson shaklda yozish.

□ **Loading/error state?**

Javob: Request kutish va xatolik holatlarini UI'da ko'rsatish.

15. React Hook Form

□ **useForm?**

Javob: Form state, submit, validation va errorsni boshqarish Hooki.

□ **register?**

Javob: Inputni form boshqaruvi va validationga ulash.

□ **handleSubmit?**

Javob: Submitni validationdan o'tkazib callbackni chaqirish.

□ **Controller?**

Javob: Controlled third-party componentlarni React Hook Form bilan ulash.

□ **watch?**

Javob: Form qiymatlarini kuzatish.

□ **reset?**

Javob: Form qiymatlarini default holatga qaytarish.

16. Zustand

□ **Zustand nima?**

Javob: React uchun yengil global state management kutubxonasi.

□ **Store nima?**

Javob: State va actionlar saqlanadigan markaziy joy.

□ **Local vs global state?**

Javob: Local yaqin componentlar uchun; global ko'p joydan kerak bo'ladigan data uchun.

□ **persist?**

Javob: Zustand state'ni storagega saqlab, reload bo'lganda tiklashga yordam beradi.

17. Advanced va Tricky

□ **Immutability?**

Javob: Object/arrayni bevosita mutate qilmasdan yangi reference yaratish prinsipi.

□ **Batching?**

Javob: React bir nechta state update'larni guruhlab, ortiqcha renderlarni kamaytirishi mumkin.

□ **StrictMode?**

Javob: Developmentda potential muammolarni aniqlash uchun qo'shimcha tekshiruvlar.

□ **Custom Hook?**

Javob: Qayta ishlatiladigan React logicni function ichiga ajratish; nomi odatda use bilan boshlanadi.

□ **Error Boundary?**

Javob: Component tree ichidagi ayrim render xatolarini ushlab fallback UI ko'rsatishga yordam beradi.

□ **Re-render va remount farqi?**

Javob: Re-render — component yana hisoblanadi; remount — eski instance olib tashlanib, yangi instance yaratiladi.

□ **useEffect faqat API uchunmi?**

Javob: Yo'q, u umumiy side effectlar uchun.

□ **useRef state o'rnini bosa oladimi?**

Javob: Ulga ta'sir qiladigan data uchun state kerak; renderga ta'sir qilmaydigan persistent qiymat uchun ref mos.

□ **Virtual DOM har doim real DOMdan tezmi?**

Javob: Buni shunday umumlashtirish noto'g'ri. React reconciliation orqali DOM update'larini samarali tashkil qiladi; natija workloadga bog'liq.

□ **State setter chaqirilganda state darhol o'zgaradimi?**

Javob: O'sha function scope ichidagi current state darhol o'zgarmaydi; update keyingi renderda aks etadi.